

PRUEBAS END TO END (E2E)

Las pruebas End-to-End (E2E) son pruebas que verifican el sistema completo, tal como lo usaria un usuario real. La idea es simple: en lugar de probar cada pieza por separado, se prueba todo junto, desde que se hace una peticion HTTP hasta que los datos llegan a PostgreSQL y regresan con una respuesta.

Lo que diferencia una prueba E2E de una prueba unitaria es que no se mockea nada. La API esta corriendo en Docker, la base de datos tiene datos reales, y el navegador (en el caso del frontend) es un Chromium real. Si algo falla, es porque realmente esta fallando en el sistema.

Para este proyecto se usaron herramientas del ecosistema Python, que resulto la opcion mas rapida de configurar sin necesidad de instalar SDKs adicionales ni tocar la solucion de .NET:

- pytest — el framework que organiza y corre los casos de prueba.
- requests — para hacer las llamadas HTTP reales a la API.
- pytest-playwright — para abrir un navegador Chromium real y probar el frontend.
- Docker Compose — para levantar todo el entorno (API + PostgreSQL + Frontend) antes de correr las pruebas.

ISSUE LINEAR: <https://linear.app/hospital-veterinario-patitas/issue/PROYECT-94/implementacion-pruebas-de-end-to-end>

GitHub de la rama: <https://github.com/lcarranzag2-ui/ProyectoAnalisisdeSistemas>

ESTRUCTURA DE ARCHIVOS

Se creo una carpeta tests/ en la raiz del proyecto (al mismo nivel que HiddenValley/) para no mezclar el codigo de pruebas con el codigo de produccion. Dentro quedaron tres archivos:

```
tests/
  conftest.py          - configuracion base, verifica que la
API este activa antes de correr
  test_reservaciones.py - 7 pruebas de API usando requests
  test_frontend.py     - 10 pruebas de UI usando Playwright +
Chromium
```

PRUEBAS DE API

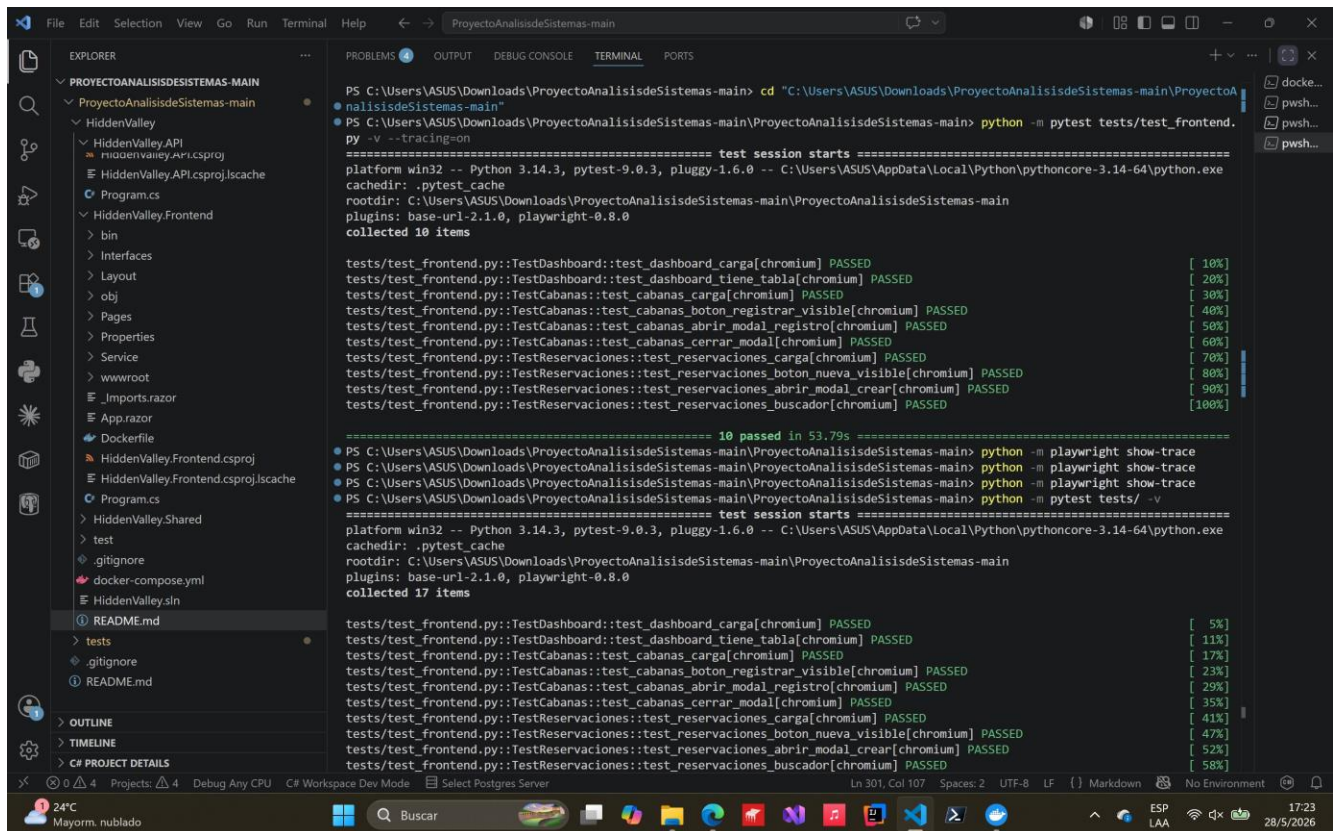
Estas pruebas hacen peticiones HTTP reales al backend en <http://localhost:7084> y verifican que cada endpoint responda con el código y los datos correctos. No hay nada simulado: la API está corriendo en Docker y los datos vienen de PostgreSQL.

Se implementaron 7 casos de prueba cubriendo los flujos principales del negocio:

test_crear_persona	POST /api/personas. Crea una persona con DPI único generado en base al timestamp para evitar duplicados entre corridas consecutivas.
test_buscar_cliente_existente	GET /api/clientes/buscar. Busca al cliente Juan Perez por teléfono y espera 200 OK con datos.
test_listar_cabanas	GET /api/cabanas. Verifica que el endpoint esté activo y devuelva la lista.
test_crear_reservacion_exitosa	POST /api/reservaciones. Flujo completo: crea una reservación con fechas futuras válidas y verifica que retorne 201 con el campo totalPagar.
test_reservacion_fechas_invalidas	POST /api/reservaciones. Manda una fecha de salida anterior a la entrada y verifica que la API la rechace con 400 BadRequest.
test_listar_reservaciones	GET /api/reservaciones. Lista todas las reservaciones y verifica 200 OK.
test_dashboard	GET /api/dashboard. Verifica que la respuesta incluya cabanasDisponibles y cabanasOcupadasHoy.

Resultados

Se corrieron las 7 pruebas con el sistema levantado en Docker. Todas pasaron. La imagen muestra la ejecución completa de las 17 pruebas (API + frontend juntas):



```
PS C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main> cd "C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoA
nalisisdeSistemas-main"
PS C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main> python -m pytest tests/test_frontend.
py -v --tracing-on
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\ASUS\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main
plugins: base-url-2.1.0, playwright-0.8.0
collected 10 items

tests/test_frontend.py::TestDashboard::test_dashboard_carga[chromium] PASSED [ 10%]
tests/test_frontend.py::TestDashboard::test_dashboard_tiene_tabla[chromium] PASSED [ 20%]
tests/test_frontend.py::TestCabinas::test_cabinas_carga[chromium] PASSED [ 30%]
tests/test_frontend.py::TestCabinas::test_cabinas_boton_registrar_visible[chromium] PASSED [ 40%]
tests/test_frontend.py::TestCabinas::test_cabinas_abrir_modal_registro[chromium] PASSED [ 50%]
tests/test_frontend.py::TestCabinas::test_cabinas_cerrar_modal[chromium] PASSED [ 60%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_carga[chromium] PASSED [ 70%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_boton_nueva_visible[chromium] PASSED [ 80%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_abrir_modal_crear[chromium] PASSED [ 90%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_buscar[chromium] PASSED [100%]

===== 10 passed in 53.79s =====
PS C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main> python -m playwright show-trace
PS C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main> python -m playwright show-trace
PS C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main> python -m playwright show-trace
PS C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main> python -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\ASUS\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ASUS\Downloads\ProyectoAnalisisdeSistemas-main\ProyectoAnalisisdeSistemas-main
plugins: base-url-2.1.0, playwright-0.8.0
collected 17 items

tests/test_frontend.py::TestDashboard::test_dashboard_carga[chromium] PASSED [ 5%]
tests/test_frontend.py::TestDashboard::test_dashboard_tiene_tabla[chromium] PASSED [ 11%]
tests/test_frontend.py::TestCabinas::test_cabinas_carga[chromium] PASSED [ 17%]
tests/test_frontend.py::TestCabinas::test_cabinas_boton_registrar_visible[chromium] PASSED [ 23%]
tests/test_frontend.py::TestCabinas::test_cabinas_abrir_modal_registro[chromium] PASSED [ 29%]
tests/test_frontend.py::TestCabinas::test_cabinas_cerrar_modal[chromium] PASSED [ 35%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_carga[chromium] PASSED [ 41%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_boton_nueva_visible[chromium] PASSED [ 47%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_abrir_modal_crear[chromium] PASSED [ 52%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_buscar[chromium] PASSED [ 58%]
```

Figura 1. Las 17 pruebas corriendo juntas con `python -m pytest tests/ -v`, resultado: 17 passed en 57.92s.

Bug encontrado durante las pruebas

Una de las pruebas mas utiles fue `test_crear_reservacion_exitosa`, que fallo con un error 500 al principio. El mensaje del servidor fue un `System.InvalidOperationException` en `ReservacionService.cs` linea 128. La causa: alguien habia dejado una linea de prueba manual (`cabana.IdCabana = 2`) en el codigo de produccion. Entity Framework no permite modificar la clave primaria de una entidad que ya esta siendo trackeada, asi que lanzaba la excepcion al intentar guardar. La prueba E2E lo detecto de inmediato con el archivo y numero de linea exactos. Se corrigio eliminando esa linea.

PRUEBAS DE FRONTEND CON PLAYWRIGHT

Las pruebas de frontend son las mas interesantes porque abren un navegador Chromium real, navegan por la aplicacion haciendo clics, llenan formularios y verifican que los elementos aparezcan como se espera. Es literalmente un robot usando la aplicacion.

Un detalle importante que se descubrio: nginx (el servidor que sirve el frontend en Docker) devuelve 404 cuando se accede directamente a rutas como `/cabinas` o `/reservaciones` porque la aplicacion es un SPA. La solucion fue hacer que Playwright

siempre navegue primero al inicio y luego use los links del menu lateral, igual que lo haria un usuario real.

Se implementaron 10 casos de prueba cubriendo las paginas principales:

test_dashboard_carga	Abre el sistema, navega a Dashboard desde el menu y verifica que aparezca el titulo de operaciones del dia.
test_dashboard_tiene_tabla	Verifica que el dashboard muestre el resumen en tiempo real del turicentro.
test_cabanas_carga	Navega a Cabanas y verifica que el titulo Gestion de Cabanas sea visible.
test_cabanas_boton_registrar_visible	Verifica que el boton Registrar Nueva Cabana aparezca en la pagina.
test_cabanas_abrir_modal_registro	Hace clic en Registrar Nueva Cabana y verifica que el modal se abra con el titulo correcto.
test_cabanas_cerrar_modal	Abre el modal y verifica que el boton Cancelar lo cierre correctamente.
test_reservaciones_carga	Navega a Reservaciones y verifica que el Panel de Reservaciones cargue.
test_reservaciones_boton_nueva_visible	Verifica que el boton Nueva Reservacion sea visible en la pagina.
test_reservaciones_abrir_modal_crear	Hace clic en Nueva Reservacion y verifica que el modal aparezca.
test_reservaciones_buscar	Escribe 'Juan' en el buscador, hace clic en Buscar y verifica que la pagina siga funcionando.

Resultados

Las 10 pruebas de frontend pasaron. Con la opcion --tracing=on se genera evidencia visual de cada accion que hizo el navegador, lo que facilita revisar exactamente que ocurrio si alguna prueba falla en el futuro:

```
PS C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main> python -m pytest tests/test_frontend.py -v --tracing-on
tests/test_frontend.py::TestCabanas::test_cabanas_cerrar_modal[chromium] PASSED [ 60%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_carga[chromium] PASSED [ 70%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_boton_nueva_visible[chromium] PASSED [ 80%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_abrir_modal_crear[chromium] PASSED [ 90%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_buscar[chromium] PASSED [100%]

===== 10 passed in 53.79s =====

PS C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main> python -m playwright show-trace
PS C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main> python -m playwright show-trace
PS C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main> python -m playwright show-trace
PS C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main> python -m pytest tests/ -v

platform win32 -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\ASUS\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main
plugins: base-url-2.1.0, playwright-0.8.0
collected 17 items

tests/test_frontend.py::TestDashboard::test_dashboard_carga[chromium] PASSED [ 5%]
tests/test_frontend.py::TestDashboard::test_dashboard_tiene_tabla[chromium] PASSED [ 11%]
tests/test_frontend.py::TestCabanas::test_cabanas_carga[chromium] PASSED [ 17%]
tests/test_frontend.py::TestCabanas::test_cabanas_boton_registrar_visible[chromium] PASSED [ 23%]
tests/test_frontend.py::TestCabanas::test_cabanas_abrir_modal_registro[chromium] PASSED [ 29%]
tests/test_frontend.py::TestCabanas::test_cabanas_cerrar_modal[chromium] PASSED [ 35%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_carga[chromium] PASSED [ 41%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_boton_nueva_visible[chromium] PASSED [ 47%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_abrir_modal_crear[chromium] PASSED [ 53%]
tests/test_frontend.py::TestReservaciones::test_reservaciones_buscar[chromium] PASSED [ 58%]
tests/test_reservaciones.py::TestPersonasVClientes::test_crear_persona PASSED [ 64%]
tests/test_reservaciones.py::TestPersonasVClientes::test_buscar_cliente_existente PASSED [ 70%]
tests/test_reservaciones.py::TestPersonasVClientes::test_listar_cabanas PASSED [ 76%]
tests/test_reservaciones.py::TestReservaciones::test_crear_reservacion_exitosa PASSED [ 82%]
tests/test_reservaciones.py::TestReservaciones::test_reservacion_fechas_invalidas PASSED [ 88%]
tests/test_reservaciones.py::TestReservaciones::test_listar_reservaciones PASSED [ 94%]
tests/test_reservaciones.py::TestReservaciones::test_dashboard PASSED [100%]

===== 17 passed in 57.92s =====

PS C:\Users\ASUS\Downloads\ProyectoAnálisisdeSistemas-main\ProyectoAnálisisdeSistemas-main>
```

Figura 2. Las 10 pruebas de frontend con resultado 10 passed en 53.79s.

Playwright Trace Viewer

tests-test-frontend-py-testreservaciones-test-reservaciones-boton-nueva-visible-chromium

Timeline: 500ms, 1.0s, 1.5s, 2.0s, 2.5s, 3.0s, 3.5s, 4.0s, 4.5s

Actions: Metadata, Action, Before, After

Filter actions

- Create page 95ms
- Navigate to "/" 308ms
- Wait for timeout 2.0s
- Click 113ms
- locator("nav a[href='reservaciones']")
- Wait for timeout 2.0s
- Expect "to_be_visible" 6ms
- locator("text=Nueva Reservación")

Locator: Call Log Errors Console Network Source Attachments

Expect "to_be_visible" locator("text=Nueva Reservación")

TIME

start: 4.6s

duration: 6ms

PARAMETERS

locator: locator("text=Nueva Reservación")

expression: "to_be_visible"

timeout: 10000

Screenshot: Panel de Reservaciones

Panel de Reservaciones

Gestión del ciclo de vida de reservas de clientes — Hidden Valley

Buscar por ID o nombre del cliente...

Buscar

Nueva Reservación

Figura 3. Playwright Trace Viewer mostrando la prueba `test_reservaciones_boton_nueva_visible`. Se puede ver cada acción del navegador con su tiempo de ejecución y el estado de la página en ese momento.

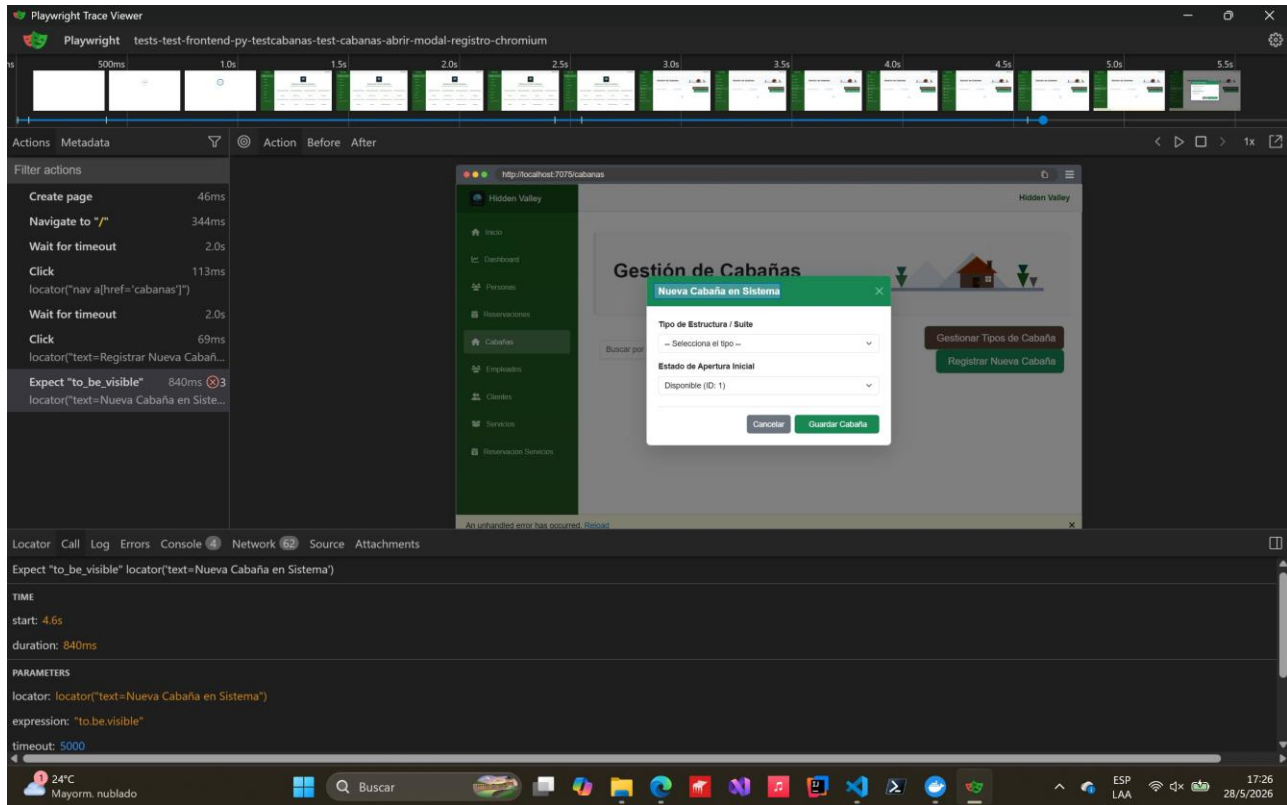


Figura 4. Playwright Trace Viewer mostrando la prueba `test_cabanas_abrir_modal_registro`. El modal Nueva Cabana en Sistema aparece abierto correctamente despues de hacer clic en el boton.

Problema de conexion frontend-API

Durante las pruebas de frontend se noto que el dashboard y otras paginas mostraban el mensaje 'No se pudo obtener la informacion del servidor. Verifique que el backend este activo.' incluso con Docker corriendo.

La causa fue que en Program.cs del proyecto Blazor WebAssembly la URL del backend estaba fija en `http://localhost:5017/`, que es el puerto que usa dotnet run en desarrollo local. Pero en Docker, la API corre en `http://localhost:7084/`. El navegador intentaba conectar al puerto equivocado y no encontraba nada.

La correccion fue cambiar esa URL en Program.cs al puerto correcto (7084). Este tipo de problema de configuracion de entorno es exactamente lo que las pruebas E2E detectan: algo que funciona en el IDE pero falla en el entorno real de Docker.

VIDEO DEMOSTRATIVO

Video de pruebas utilizando la tecnología Playwright donde se muestra el funcionamiento de el proyecto lo utilizaria un usuario:

<https://drive.google.com/file/d/1S3RREm-Wbc22cVSkcvxukg6VmbCIQgRZ/view?usp=sharing>

COMO CORRER LAS PRUEBAS

Con Docker levantado y los datos base insertados en la BD, desde la raiz del proyecto:

```
# Solo las pruebas de API
python -m pytest tests/test_reservaciones.py -v
```

```
# Solo las pruebas de frontend
python -m pytest tests/test_frontend.py -v
```

```
# Frontend con el navegador visible (util para ver que esta haciendo)
python -m pytest tests/test_frontend.py -v --headed
```

```
# Todas las pruebas juntas
python -m pytest tests/ -v
```

```
# Generar reporte HTML
python -m pytest tests/ -v --html=reporte.html
```